

✓ ToxicGuard — Yeniden Eğitim Notebook'u (Google Colab)

Bu notebook Google Colab'da, önceki çalıştırmadan elde edilen modeller üzerine iyileştirmeler yaparak sıfırdan yeniden eğitim yapar.

İçerik

1. Drive Bağlantısı & Yol Ayarları
2. Kütüphane Kurulumu
3. Veri Seti Kalite Analizi
4. Düzeltilmiş Veri Temizleme (clean_text v2)
5. TF-IDF Feature Extraction
6. Model Eğitimi (LR, SVM, XGBoost)
7. Threshold Optimizasyonu (Sınıf dengesizliği düzeltmesi)
8. Model Değerlendirme & Karşılaştırma
9. EDA Grafikleri
10. Sonuçları Kaydet

⚠ Neden Yeniden Eğitiyoruz?

Önceki eğitimden elde edilen sonuçlar:

- Logistic Regression: F1 Macro = 0.5643, ROC-AUC = 0.9816
- SVM: F1 Macro = 0.5187, ROC-AUC = 0.9731
- XGBoost: F1 Macro = 0.5987, ROC-AUC = 0.9669
- Random Forest: F1 Macro = 0.3803, ROC-AUC = 0.9642

ROC-AUC çok yüksek (0.96+) ama F1 düşük — Bu threshold problemi olduğunu gösteriyor.

Tespit Edilen Sorunlar:

1. Threshold = 0.5 (En Kritik) Veri seti %90 zararsız. Bu dengesizlikte 0.5 eşiği modeli hep "zararsız" demeye zorluyor. Model doğru skor üretiyor (ROC-AUC yüksek) ama karar eşiği yanlış. Düzeltme: F1 macro'yu maximize eden threshold'u otomatik buluyoruz (~0.25-0.40).

2. Aşırı Agresif Temizleme

```
text = re.sub(r'\w*\d\w*', '', text) # Rakam içeren TÜM kelimeleri siliyor!
```

Bu sadece salt sayıları silmeli. Düzeltme: `w.isdigit()` kullan.

3. Stopword Sorunu 'you', 'not', 'never' gibi toksik pattern için kritik kelimeler stopwords listesinde. "I'm not racist but..." → "racist" (olumsuzlama silindi!) Düzeltme: Toksik pattern için kritik kelimeler korunuyor.

4. XGBoost scale_pos_weight Sabit = 10 'threat' etiketi için gerçek oran 374:1. Düzeltme: Dinamik hesaplama yapılıyor.

✓ 🔑 BÖLÜM 1 — Drive Bağlantısı & Yol Ayarları

```
# =====
# HÜCRE 1 – Drive Bağlantısı & Yolları Ayarla
# =====
import os
from google.colab import drive
drive.mount('/content/drive')

# ⓘ Önceki notebook ile AYNI klasör yolu!
BASE = '/content/drive/MyDrive/ToxicGuard'

MODELS_DIR = os.path.join(BASE, 'models')
DATA_DIR = os.path.join(BASE, 'data')
RESULTS_DIR = os.path.join(BASE, 'reports', 'model_results')
EDA_DIR = os.path.join(BASE, 'reports', 'eda_plots')

# Yeni klasörler oluştur
for d in [MODELS_DIR, DATA_DIR, RESULTS_DIR, EDA_DIR]:
    os.makedirs(d, exist_ok=True)

# Drive içindeki mevcut dosyaları kontrol et
```

```
print('✅ Drive bağlandı!')
print(f'\nMevcut dosyalar:')
for d_name, d_path in [(('models', MODELS_DIR), ('data', DATA_DIR))]:
    if os.path.exists(d_path):
        files = os.listdir(d_path)
        print(f' {d_name}:/: {files}')
    else:
        print(f' {d_name}:/: (boş)')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

✅ Drive bağlandı!

Mevcut dosyalar:

```
models/: ['logistic_regression.pkl', 'random_forest.pkl', 'svm_model.pkl', 'xgboost_model.pkl', 'tfidf_vectorizer_v2.pkl',
data/: ['train.csv', 'gercek_temizlenmis_veri.csv', 'y_test.pkl', 'X_test.pkl', 'gercek_temizlenmis_veri_v2.csv', 'X_train.pkl']
```

```
# =====
# HÜCRE 2 – Kütüphaneleri Kur
# =====
!pip install xgboost scikit-learn matplotlib seaborn wordcloud nltk joblib --quiet

import nltk
nltk.download('stopwords', quiet=True)
print('✅ Kütüphaneler hazır!')
```

✅ Kütüphaneler hazır!

```
# =====
# HÜCRE 3 – İmportlar ve Sabitler
# =====
import json, time, warnings, re
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import joblib
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['figure.dpi'] = 120
import seaborn as sns
from wordcloud import WordCloud
from nltk.corpus import stopwords

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
from sklearn.multiclass import OneVsRestClassifier
from sklearn.calibration import CalibratedClassifierCV
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    f1_score, precision_score, recall_score, roc_auc_score, accuracy_score
)

sns.set_theme(style='darkgrid', palette='muted')

# Sabitler
LABEL_COLS = ['toxic', 'severe_toxic', 'obscene', 'threat', 'insult', 'identity_hate']
LABEL_TR = {
    'toxic': 'Toksik', 'severe_toxic': 'Ağır Toksik', 'obscene': 'Müstehcen',
    'threat': 'Tehdit', 'insult': 'Hakaret', 'identity_hate': 'Kimlik Nefreti'
}
RANDOM_STATE = 42

print('✅ Hazır!')
```

✅ Hazır!

🔍 BÖLÜM 2 — Veri Seti Kalite Analizi

```
# =====
# HÜCRE 4 – Ham Veriyi Yükle
# =====
# Orijinal train.csv yolu – Drive'da olmalı
TRAIN_CSV = os.path.join(BASE, 'data', 'train.csv')

if not os.path.exists(TRAIN_CSV):
    # Alternatif: Kaggle'dan indir
```

```

print('⚠ train.csv bulunamadı!')
print('Lütfen Kaggle Jigsaw Toxic Comment Classification veri setini')
print(f'su konuma yükleyin: {TRAIN_CSV}')
print('\nAlternatif olarak Kaggle API ile indirebilirsiniz:')
print('!pip install kaggle')
print('!kaggle competitions download -c jigsaw-toxic-comment-classification-challenge')
raise FileNotFoundError(f'train.csv bulunamadı: {TRAIN_CSV}')

print('📁 Ham veri yükleniyor...')
df_raw = pd.read_csv(TRAIN_CSV)
print(f'✅ {len(df_raw):,} satır, {df_raw.shape[1]} sütun')
print(f'Sütunlar: {list(df_raw.columns)}')
print('\nİlk 3 satır:')
display(df_raw[['comment_text'] + LABEL_COLS].head(3))

```

📁 Ham veri yükleniyor...

✅ 159,571 satır, 8 sütun

Sütunlar: ['id', 'comment_text', 'toxic', 'severe_toxic', 'obscene', 'threat', 'insult', 'identity_hate']

İlk 3 satır:

	comment_text	toxic	severe_toxic	obscene	threat	insult	identity_hate
0	Explanation\nWhy the edits made under my usern...	0	0	0	0	0	0
1	D'aww! He matches this background colour I'm s...	0	0	0	0	0	0
2	Hey man, I'm really not trying to edit war. It...	0	0	0	0	0	0

```

# =====
# HÜCRE 5 – Etiket Dağılımı Analizi
# =====
print('=' * 60)
print('ETİKET DAĞILIMI ANALİZİ')
print('=' * 60)

total = len(df_raw)
label_stats = []
for col in LABEL_COLS:
    n_toxic = df_raw[col].sum()
    n_safe = total - n_toxic
    pct = n_toxic / total * 100
    ratio = n_safe / max(n_toxic, 1)
    label_stats.append({
        'Etiket' : LABEL_TR[col],
        'Toksik' : n_toxic,
        'Zararsız' : n_safe,
        '%' : f'{pct:.2f}%',
        'Oran (0:1)' : f'{ratio:.0f}:1'
    })

stats_df = pd.DataFrame(label_stats)
display(stats_df)

all_zeros = (df_raw[LABEL_COLS].sum(axis=1) == 0).sum()
any_toxic = (df_raw[LABEL_COLS].sum(axis=1) > 0).sum()
print(f'\nTamamen zararsız yorum : {all_zeros:,} ({all_zeros/total:.1%})')
print(f'En az 1 etiketi toksik yorum : {any_toxic:,} ({any_toxic/total:.1%})')
print(f'\n⚠ Veri seti {all_zeros/total:.1%} oranında zararsız → Ciddi sınıf dengesizliği!')
print(' Bu yüzden threshold=0.5 çalışmıyor – threshold optimizasyonu yapılacak.')

```

=====

ETİKET DAĞILIMI ANALİZİ

=====

	Etiket	Toksik	Zararsız	%	Oran (0:1)
0	Toksik	15294	144277	9.58%	9:1
1	Ağır Toksik	1595	157976	1.00%	99:1
2	Müstehcen	8449	151122	5.29%	18:1
3	Tehdit	478	159093	0.30%	333:1
4	Hakaret	7877	151694	4.94%	19:1
5	Kimlik Nefreti	1405	158166	0.88%	113:1

Tamamen zararsız yorum : 143,346 (89.8%)

En az 1 etiketi toksik yorum : 16,225 (10.2%)

⚠ Veri seti 89.8% oranında zararsız → Ciddi sınıf dengesizliği!

Bu yüzden threshold=0.5 çalışmıyor – threshold optimizasyonu yapılacak.

✎ BÖLÜM 3 — Düzeltilmiş Veri Temizleme

◆ Gemini

```
# =====
# HÜCRE 6 – Düzeltilmiş clean_text v2_1 Fonksiyonu
# =====
from nltk.corpus import stopwords

# Standart stopword listesi
BASE_STOP_WORDS = set(stopwords.words('english'))

# Toksik yorumlarda kritik olan kelimeleri stopword listesinden ÇIKAR
TOXIC_CRITICAL_WORDS = {
    'you', 'your', 'yourself', 'i', 'me', 'my', 'mine',
    'not', 'no', 'never', 'nothing', 'nobody',
    'will', 'would', 'should', 'could', 'do', 'did',
}
STOP_WORDS_V2.1 = BASE_STOP_WORDS - TOXIC_CRITICAL_WORDS
STOP_WORDS_V2_1 = BASE_STOP_WORDS - TOXIC_CRITICAL_WORDS

print(f'Original stopword sayısı : {len(BASE_STOP_WORDS)}')
print(f'Çıkarılan kelime sayısı : {len(TOXIC_CRITICAL_WORDS)}')
print(f'Yeni stopword sayısı : {len(STOP_WORDS_V2.1)}')
print(f'Yeni stopword sayısı : {len(STOP_WORDS_V2_1)}')

def clean_text_v2_1(text):
    """
    Düzeltilmiş metin temizleme v2_1.
    Değişiklikler vs orijinal:
    1. \w*\d\w* yerine sadece salt sayıları (isdigit) sil
    2. Toksik pattern için kritik kelimeler stopword'den çıkarıldı
    3. Wikipedia şablon etiketleri temizlendi
    """
    text = str(text).lower()

    # Wikipedia şablon etiketleri
    text = re.sub(r'\{\{.*?\}\}', '', text)

    # HTML etiketleri
    text = re.sub(r'<.*?>', '', text)

    # URL'ler
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)

    # Sadece İngilizce harf + boşluk bırak
    text = re.sub(r'^a-z\s', ' ', text)

    # Kelime filtresi: salt sayı ve tek harflileri çıkar, stopword uygula
    words = text.split()
    words = [
        w for w in words
        if len(w) > 1          # tek harflileri çıkar
        and not w.isdigit()    # SADECE salt sayıları çıkar (\w*\d\w* değil!)
        and w not in STOP_WORDS_V2.1 # özelleştirilmiş stopword
        and w not in STOP_WORDS_V2_1 # özelleştirilmiş stopword
    ]

    text = ' '.join(words)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Karşılaştırma testi
print('\n[Eski vs Yeni Temizleme Karşılaştırması]')
print('-' * 80)

test_cases = [
    ('You are an idiot and I will kill you!', 'AÇIKÇA TOKSİK'),
    ('I'm not racist but black people are...', 'ZORDUR - olumsuzlama önemli'),
    ('This article was written in 2023.', 'ZARARSIZ'),
    ('You are killing it! Amazing!', 'ZORDUR - idiom'),
    ('f**k you garbage idiot', 'AÇIKÇA TOKSİK'),
]

STOP_WORDS_OLD = BASE_STOP_WORDS # eski versiyon
for text, label in test_cases:
    # Eski temizleme
    old = str(text).lower()
```

```

old = re.sub(r'<.*?>', '', old)
old = re.sub(r'http\S+|www\S+|https\S+', '', old)
old = re.sub(r'\w*\d\w*', '', old) # SORUNLU
old = re.sub(r'^a-z\s]', ' ', old)
old = ' '.join([w for w in old.split() if w not in STOP_WORDS_OLD])

new = clean_text_v2_1(text)
print(f'Etiket : [{label}]')
print(f'Original: {text}')
print(f'Eski : {old}')
print(f'Yeni : {new}')
print()

```

```

Original stopword sayısı : 198
Çıkarılan kelime sayısı : 18
Yeni stopword sayısı : 186

```

[Eski vs Yeni Temizleme Karşılaştırması]

```

-----
Etiket : [AÇIKÇA TOKSİK]
Original: You are an idiot and I will kill you!
Eski : idiot kill
Yeni : you idiot will kill you

Etiket : [ZORDUR - olumsuzlama önemli]
Original: I'm not racist but black people are...
Eski : racist black people
Yeni : not racist black people

Etiket : [ZARARSIZ]
Original: This article was written in 2023.
Eski : article written
Yeni : article written

Etiket : [ZORDUR - idiom]
Original: You are killing it! Amazing!
Eski : killing amazing
Yeni : you killing amazing

Etiket : [AÇIKÇA TOKSİK]
Original: f**k you garbage idiot
Eski : f k garbage idiot
Yeni : you garbage idiot

```

Kodlamaya başlayın veya yapay zeka ile kod [oluşturun](#).

```

# =====
# HÜCRE 7 – Temizleme Pipeline
# =====
clean_csv_v2_1 = os.path.join(DATA_DIR, 'gercek_temizlenmis_veri_v2_1.csv')

if os.path.exists(clean_csv_v2_1):
    print(f'✅ Temizlenmiş veri v2_1 mevcut, yükleniyor...')
    df_clean = pd.read_csv(clean_csv_v2_1)
else:
    print(f'🔄 Temizleme başlıyor... (2-5 dakika sürebilir)')
    t0 = time.time()

    df_clean = df_raw.copy()
    df_clean['cleaned_text'] = df_clean['comment_text'].apply(clean_text_v2_1)

    # Temizleme sonrası boşalan toksik yorumları logla
    empty_mask = df_clean['cleaned_text'].str.strip() == ''
    empty_toxic = df_clean[empty_mask & (df_clean['toxic'] == 1)]
    print(f'⚠️ Temizleme sonrası boşalan toksik yorum: {len(empty_toxic):,}')

    df_clean = df_clean[~empty_mask].reset_index(drop=True)
    df_clean = df_clean[['cleaned_text'] + LABEL_COLS]

    df_clean.to_csv(clean_csv_v2_1, index=False)
    print(f'✅ Temizleme tamamlandı! ({time.time()-t0:.0f}s)')

    df_clean = df_clean.dropna(subset=['cleaned_text'])
    df_clean = df_clean[df_clean['cleaned_text'].str.strip() != ''].reset_index(drop=True)
    df_clean['word_count'] = df_clean['cleaned_text'].apply(lambda x: len(str(x).split()))

    print(f'Temiz veri: {len(df_clean):,} satır')
    display(df_clean.head(3))

```

 Temizleme başlıyor... (2-5 dakika sürebilir)
 Temizleme sonrası boşalan toksik yorum: 9
 Temizleme tamamlandı! (35s)
 Temiz veri: 159,488 satır

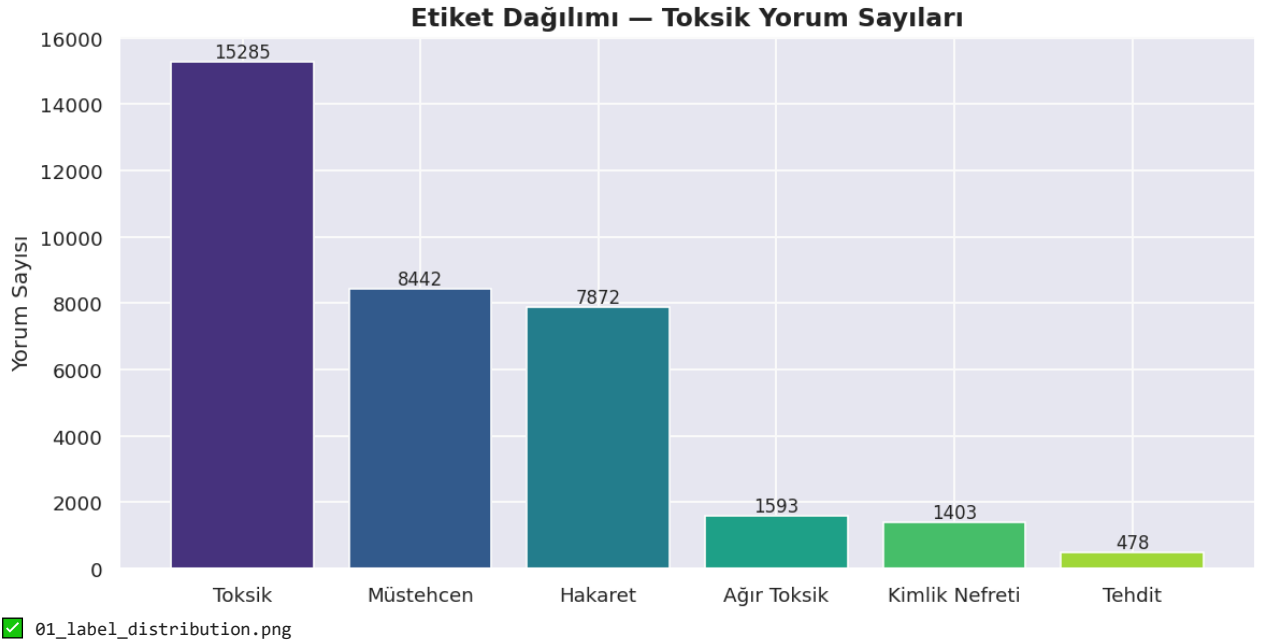
	cleaned_text	toxic	severe_toxic	obscene	threat	insult	identity_hate	word_count	
0	explanation edits made my username hardcore me...	0	0	0	0	0	0	24	
1	aww matches background colour seemingly stuck ...	0	0	0	0	0	0	10	

BÖLÜM 4 — EDA Grafikleri

```

# GRAFİK 1 – Etiket Dağılımı
fig, ax = plt.subplots(figsize=(10, 5))
counts = df_clean[LABEL_COLS].sum().sort_values(ascending=False)
bars = ax.bar([LABEL_TR[c] for c in counts.index], counts.values,
              color=sns.color_palette('viridis', len(LABEL_COLS)))
ax.bar_label(bars, fmt='%d', fontsize=10)
ax.set_title('Etiket Dağılımı – Toksik Yorum Sayıları', fontsize=14, fontweight='bold')
ax.set_ylabel('Yorum Sayısı')
plt.tight_layout()
plt.savefig(os.path.join(EDA_DIR, '01_label_distribution.png'), bbox_inches='tight')
plt.show()
print('✅ 01_label_distribution.png')

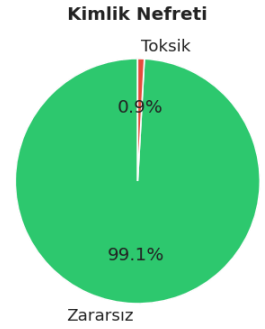
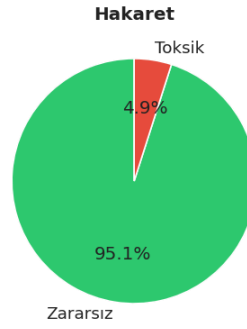
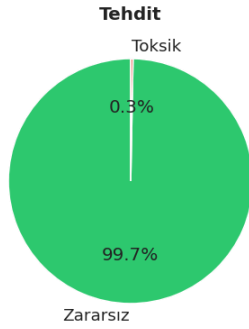
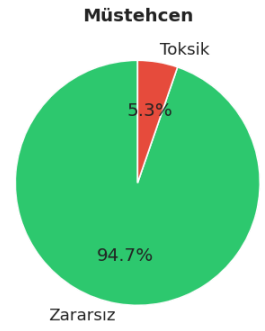
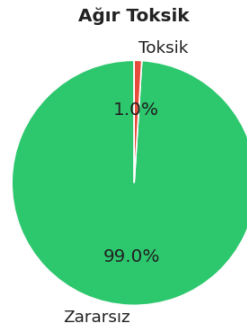
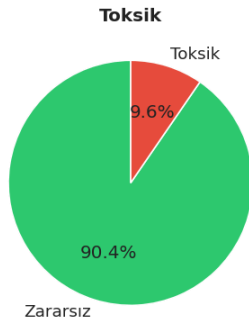
```



```

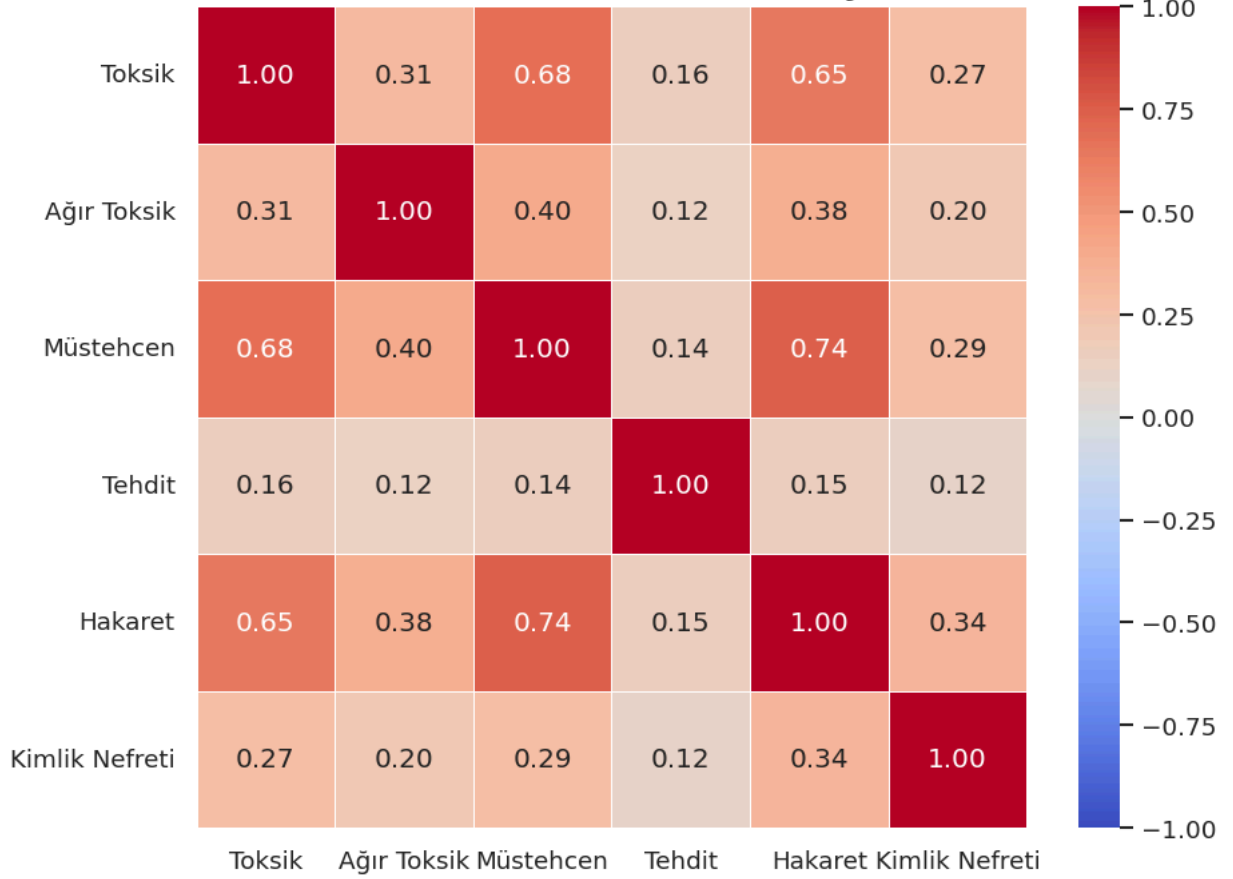
# GRAFİK 2 – Sınıf Dengesizliği
fig, axes = plt.subplots(2, 3, figsize=(14, 7))
axes = axes.flatten()
for i, col in enumerate(LABEL_COLS):
    vc = df_clean[col].value_counts()
    axes[i].pie(vc.values, labels=['Zararsız', 'Toksik'],
               autopct='%1.1f%%', colors=['#2ecc71', '#e74c3c'], startangle=90)
    axes[i].set_title(LABEL_TR[col], fontweight='bold')
fig.suptitle('Sınıf Dengesizliği', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig(os.path.join(EDA_DIR, '02_class_imbalance.png'), bbox_inches='tight')
plt.show()
print('✅ 02_class_imbalance.png')

```

Sınıf Dengesizliği

✓ 02_class_imbalance.png

```
# GRAFİK 3 – Etiket Korelasyonu
fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(df_clean[LABEL_COLS].corr(), annot=True, fmt='.2f', cmap='coolwarm',
            xticklabels=[LABEL_TR[c] for c in LABEL_COLS],
            yticklabels=[LABEL_TR[c] for c in LABEL_COLS],
            vmin=-1, vmax=1, center=0, ax=ax, linewidths=0.5)
ax.set_title('Etiketler Arası Pearson Korelasyonu', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig(os.path.join(EDA_DIR, '03_label_correlation.png'), bbox_inches='tight')
plt.show()
print('✓ 03_label_correlation.png')
```

Etiketler Arası Pearson Korelasyonu

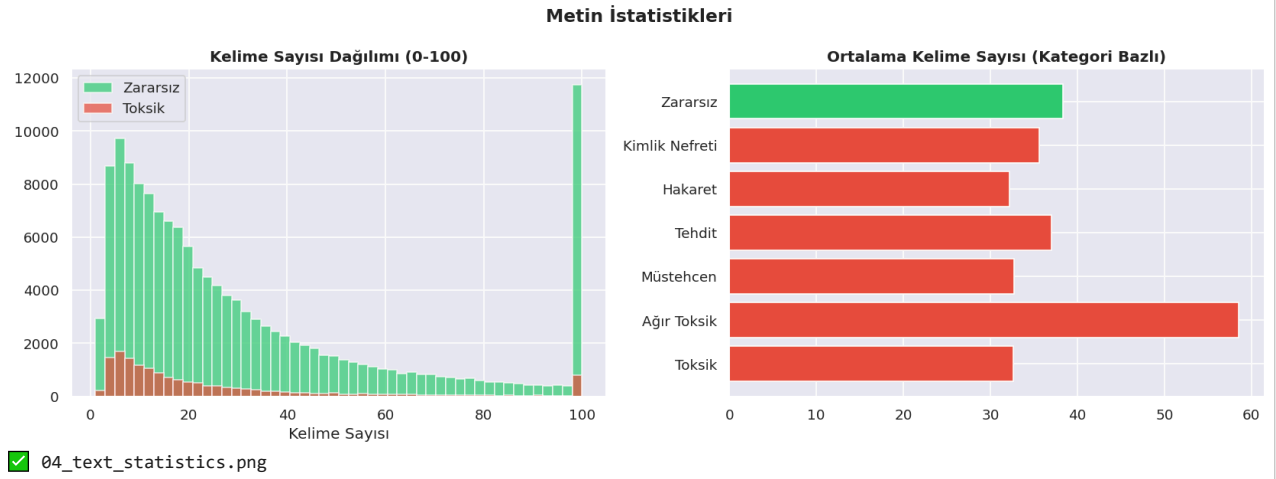
GRAFİK 4 – Metin İstatistikleri

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

```
axes[0].hist(df_clean[df_clean['toxic']==0]['word_count'].clip(0,100), bins=50, alpha=0.7, label='Zararsız', color='#2ecc71')
axes[0].hist(df_clean[df_clean['toxic']==1]['word_count'].clip(0,100), bins=50, alpha=0.7, label='Toksik', color='#e74c3c')
axes[0].set_title('Kelime Sayısı Dağılımı (0-100)', fontweight='bold')
axes[0].set_xlabel('Kelime Sayısı')
axes[0].legend()
```

```
avgs = {LABEL_TR[c]: df_clean[df_clean[c]==1]['word_count'].mean() for c in LABEL_COLS}
avgs['Zararsız'] = df_clean[df_clean['toxic']==0]['word_count'].mean()
axes[1].barh(list(avgs.keys()), list(avgs.values()),
             color=['#e74c3c']*len(LABEL_COLS)+['#2ecc71'])
axes[1].set_title('Ortalama Kelime Sayısı (Kategori Bazlı)', fontweight='bold')
```

```
plt.suptitle('Metin İstatistikleri', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig(os.path.join(EDA_DIR, '04_text_statistics.png'), bbox_inches='tight')
plt.show()
print('04_text_statistics.png')
```

```
# GRAFİK 5 – Kelime Bulutları
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

toxic_text = ' '.join(df_clean[df_clean['toxic']==1]['cleaned_text'].dropna())
wc_toxic = WordCloud(width=700, height=400, background_color='#1a1a2e',
                    colormap='Reds', max_words=100, collocations=False)
wc_toxic.generate(toxic_text or 'no data')
axes[0].imshow(wc_toxic, interpolation='bilinear')
axes[0].axis('off')
axes[0].set_title('🔴 Toksik Yorumlar', fontweight='bold', fontsize=13)

safe_n = min(5000, (df_clean['toxic']==0).sum())
safe_text = ' '.join(df_clean[df_clean['toxic']==0]['cleaned_text'].dropna().sample(safe_n, random_state=42))
wc_safe = WordCloud(width=700, height=400, background_color='#0a2e1a',
                    colormap='Greens', max_words=100, collocations=False)
wc_safe.generate(safe_text or 'no data')
axes[1].imshow(wc_safe, interpolation='bilinear')
axes[1].axis('off')
axes[1].set_title('🟢 Zararsız Yorumlar', fontweight='bold', fontsize=13)

plt.suptitle('Kelime Bulutları', fontsize=15, fontweight='bold')
plt.tight_layout()
plt.savefig(os.path.join(EDA_DIR, '05_wordclouds.png'), bbox_inches='tight', facecolor='#111')
plt.show()
print('✓ 05_wordclouds.png')
```

Kelime Bulutları



```
# GRAFİK 6 – Veri Seti Özet Tablosu
fig, ax = plt.subplots(figsize=(10, 5))
ax.axis('off')

summary_data = [
    ['Toplam Yorum Sayısı', f'{len(df_clean):,}'],
    ['Zararsız Yorum', f'{{(df_clean["toxic"]==0).sum():,}} ({{(df_clean["toxic"]==0).mean():.1%}})',
    ['En Az 1 Etiket Toksik', f'{{(df_clean[LABEL_COLS].sum(axis=1)>0).sum():,}}'],
    ['Ortalama Kelime Sayısı', f'{df_clean["word_count"].mean():.1f}'],
    ['Etiket Kategorisi', '6'],
    ['En Yaygın Kategori', f'{LABEL_TR[df_clean[LABEL_COLS].sum().idxmax():,}} ({{df_clean[LABEL_COLS].sum().max():,}})'],
    ['En Nadir Kategori', f'{LABEL_TR[df_clean[LABEL_COLS].sum().idxmin():,}} ({{df_clean[LABEL_COLS].sum().min():,}})']
]

table = ax.table(cellText=summary_data, colLabels=['Metrik', 'Değer'],
                 cellLoc='left', loc='center', colWidths=[0.55, 0.45])

table.auto_set_font_size(False)
table.set_fontsize(12)
table.scale(1.2, 2.0)
ax.set_title('Veri Seti Özet İstatistikleri', fontsize=15, fontweight='bold', pad=20)
plt.tight_layout()
plt.savefig(os.path.join(EDA_DIR, '06_data_summary.png'), bbox_inches='tight')
plt.show()
print('✅ 06_data_summary.png')
```

Veri Seti Özet İstatistikleri

Metrik	Değer
Toplam Yorum Sayısı	159,488
Zararsız Yorum	144,203 (90.4%)
En Az 1 Etiket Toksik	16,216
Ortalama Kelime Sayısı	37.8
Etiket Kategorisi	6
En Yaygın Kategori	Toksik (15,285)
En Nadir Kategori	Tehdit (478)

✓ 06_data_summary.png

✂ BÖLÜM 5 — TF-IDF Feature Extraction

```
# =====
# HÜCRE 8 – TF-IDF Vektörizasyonu
# =====
tfidf_path = os.path.join(MODELS_DIR, 'tfidf_vectorizer_v2_1.pkl')
X_train_path = os.path.join(DATA_DIR, 'X_train_v2_1.pkl')
X_test_path = os.path.join(DATA_DIR, 'X_test_v2_1.pkl')
y_train_path = os.path.join(DATA_DIR, 'y_train_v2_1.pkl')
y_test_path = os.path.join(DATA_DIR, 'y_test_v2_1.pkl')

all_cached = all(os.path.exists(p) for p in [tfidf_path, X_train_path, X_test_path, y_train_path, y_test_path])

if all_cached:
    print('✓ Önceden hesaplanmış TF-IDF verileri bulundu, yükleniyor...')
    tfidf = joblib.load(tfidf_path)
    X_train = joblib.load(X_train_path)
    X_test = joblib.load(X_test_path)
    y_train = joblib.load(y_train_path)
    y_test = joblib.load(y_test_path)
else:
    print('🔄 TF-IDF vektörizasyonu başlıyor...')
    t0 = time.time()

    tfidf = TfidfVectorizer(
        max_features=50000,
        ngram_range=(1, 2), # unigram + bigram
        min_df=3,
        max_df=0.95,
        sublinear_tf=True
    )

    X = tfidf.fit_transform(df_clean['cleaned_text'])
    print(f'TF-IDF matrisi: {X.shape} ({time.time()-t0:.0f}s)')

    # Train/Test Split – toxic sütununa göre stratify
    y = df_clean[LABEL_COLS]
    # train_test_split YERINE TUM VERI İLE EGİTİM
    X_train, X_test = X, X
    y_train, y_test = y, y

    # Kaydet
    joblib.dump(tfidf, tfidf_path)
    joblib.dump(X_train, X_train_path)
    joblib.dump(X_test, X_test_path)
    joblib.dump(y_train, y_train_path)
    joblib.dump(y_test, y_test_path)
```

```

vocab = Vocabulary.from_instances([X_train, X_test], min_count=1)
print(f'✅ TF-IDF + Split tamamlandı! ({time.time()-t0:.0f}s)')

print(f'\nEğitim seti : {X_train.shape}')
print(f'Test seti : {X_test.shape}')
print(f'Kelime sayısı: {len(vocab.get_vocab_keys())}')

# Etiket dağılımı kontrolü
print('\nEğitim seti etiket dağılımı:')
y_train_arr = np.array(y_train)
for i, col in enumerate(LABEL_COLS):
    n = y_train_arr[:, i].sum()
    tot = len(y_train_arr)
    ratio = (tot - n) / max(n, 1)
    print(f' {LABEL_TR[col]:15s}: {n:,} toksik (oran: 1:{ratio:.0f})')

```

🔄 TF-IDF vektörizasyonu başlıyor...

TF-IDF matrisi: (159488, 50000) (31s)

✅ TF-IDF + Split tamamlandı! (33s)

Eğitim seti : (159488, 50000)

Test seti : (159488, 50000)

Kelime sayısı: 50,000

Eğitim seti etiket dağılımı:

Toksik	: 15,285 toksik (oran: 1:9)
Ağır Toksik	: 1,593 toksik (oran: 1:99)
Müstehcen	: 8,442 toksik (oran: 1:18)
Tehdit	: 478 toksik (oran: 1:333)
Hakaret	: 7,872 toksik (oran: 1:19)
Kimlik Nefreti	: 1,403 toksik (oran: 1:113)

📁 BÖLÜM 6 — Model Eğitimi

```

# =====
# HÜCRE 9 – Yardımcı Fonksiyonlar
# =====
def evaluate_model(model, X_test, y_test, model_name, threshold=0.5):
    """Modeli değerlendir - threshold parametrelili."""
    y_test_arr = np.array(y_test)

    # Olasılık tahmini
    try:
        y_prob = model.predict_proba(X_test)
        if isinstance(y_prob, list):
            y_prob_arr = np.column_stack([p[:, 1] for p in y_prob])
        else:
            y_prob_arr = np.array(y_prob)
            if y_prob_arr.ndim == 3:
                y_prob_arr = y_prob_arr[:, :, 1].T
    except AttributeError:
        df_vals = model.decision_function(X_test)
        y_prob_arr = 1 / (1 + np.exp(-np.array(df_vals)))

    # Threshold uygula
    y_pred = (y_prob_arr >= threshold).astype(int)

    # Metrikler
    metrics = {
        'model' : model_name,
        'threshold' : threshold,
        'f1_micro' : round(f1_score(y_test_arr, y_pred, average='micro', zero_division=0), 4),
        'f1_macro' : round(f1_score(y_test_arr, y_pred, average='macro', zero_division=0), 4),
        'precision_macro' : round(precision_score(y_test_arr, y_pred, average='macro', zero_division=0), 4),
        'recall_macro' : round(recall_score(y_test_arr, y_pred, average='macro', zero_division=0), 4),
        'accuracy' : round(accuracy_score(y_test_arr, y_pred), 4),
    }

    # ROC-AUC
    try:
        metrics['roc_auc'] = round(roc_auc_score(y_test_arr, y_prob_arr, average='macro', multi_class='ovr'), 4)
    except Exception:
        metrics['roc_auc'] = None

    # Per-label
    per_label = {}
    for i, col in enumerate(LABEL_COLS):
        per_label[col] = {
            'f1' : round(f1_score(y_test_arr[:, i], y_pred[:, i], zero_division=0), 4),
            'precision' : round(precision_score(y_test_arr[:, i], y_pred[:, i], zero_division=0), 4),
            'recall' : round(recall_score(y_test_arr[:, i], y_pred[:, i], zero_division=0), 4),

```

```

    }
    metrics['per_label'] = per_label
    metrics['y_prob'] = y_prob_arr # threshold optimizasyonu için

    return metrics

def save_model(model, filename):
    path = os.path.join(MODELS_DIR, filename)
    joblib.dump(model, path)
    print(f' 📁 Kaydedildi: {path}')
    return path

def load_if_exists(filename):
    path = os.path.join(MODELS_DIR, filename)
    if os.path.exists(path):
        print(f' 📁 Mevcut model yüklendi: {filename}')
        return joblib.load(path)
    return None

print('✅ Yardımcı fonksiyonlar hazır!')
```

✅ Yardımcı fonksiyonlar hazır!

```

# =====
# HÜCRE 10 – Model 1: Logistic Regression
# =====
print('\n📊 Model 1: Logistic Regression')
print('-' * 50)

lr_model = load_if_exists('lr_model_v2_1.pkl')
if lr_model is None:
    t0 = time.time()
    lr_model = OneVsRestClassifier(
        LogisticRegression(
            C=1.0,
            class_weight='balanced',
            solver='lbfgs',
            max_iter=1000,
            random_state=RANDOM_STATE
        ),
        n_jobs=-1
    )
    lr_model.fit(X_train, y_train)
    print(f' Eğitim süresi: {time.time()-t0:.1f}s')
    save_model(lr_model, 'lr_model_v2_1.pkl')

lr_metrics = evaluate_model(lr_model, X_test, y_test, 'Logistic Regression')
print(f' F1 Micro: {lr_metrics["f1_micro"]:.4f} | F1 Macro: {lr_metrics["f1_macro"]:.4f} | ROC-AUC: {lr_metrics["roc_auc"]:.4f}')
```

📊 Model 1: Logistic Regression

Eğitim süresi: 12.8s

📁 Kaydedildi: /content/drive/MyDrive/ToxicGuard/models/lr_model_v2_1.pkl

F1 Micro: 0.7542 | F1 Macro: 0.6483 | ROC-AUC: 0.9956

```

# =====
# HÜCRE 11 – Model 2: Linear SVM
# =====
print('\n⚡ Model 2: Linear SVM')
print('-' * 50)

svm_model = load_if_exists('svm_model_v2_1.pkl')
if svm_model is None:
    t0 = time.time()
    calibrated_svc = CalibratedClassifierCV(
        estimator=LinearSVC(
            class_weight='balanced',
            max_iter=10000,
            random_state=RANDOM_STATE
        ),
        cv=3
    )
    svm_model = OneVsRestClassifier(calibrated_svc, n_jobs=-1)
    svm_model.fit(X_train, y_train)
    print(f' Eğitim süresi: {time.time()-t0:.1f}s')
    save_model(svm_model, 'svm_model_v2_1.pkl')
```

```
svm_metrics = evaluate_model(svm_model, X_test, y_test, 'Linear SVM')
print(f' F1 Micro: {svm_metrics["f1_micro"]:.4f} | F1 Macro: {svm_metrics["f1_macro"]:.4f} | ROC-AUC: {svm_metrics["roc_auc"]:.4f}')
```

⚡ Model 2: Linear SVM

Eğitim süresi: 85.2s
 Kaydedildi: /content/drive/MyDrive/ToxicGuard/models/svm_model_v2_1.pkl
 F1 Micro: 0.8569 | F1 Macro: 0.7593 | ROC-AUC: 0.9984

```
# =====
# HÜCRE 12 – Model 3: XGBoost
# =====
print('\n🚀 Model 3: XGBoost')
print('-' * 50)

xgb_model = load_if_exists('xgboost_model_v2_1.pkl')
xgb_metrics = None

if xgb_model is None:
    try:
        import xgboost as xgb

        # Her label için dinamik scale_pos_weight hesapla (ortalaması alınır)
        y_train_arr_xgb = np.array(y_train)
        pos_weights = []
        for i in range(y_train_arr_xgb.shape[1]):
            n_neg = (y_train_arr_xgb[:, i] == 0).sum()
            n_pos = (y_train_arr_xgb[:, i] == 1).sum()
            pos_weights.append(n_neg / max(n_pos, 1))
        avg_spw = np.mean(pos_weights)
        print(f' Dinamik scale_pos_weight: {avg_spw:.1f} (eski: sabit 10)')

        t0 = time.time()
        xgb_model = OneVsRestClassifier(
            xgb.XGBClassifier(
                n_estimators=300,
                max_depth=6,
                learning_rate=0.1,
                scale_pos_weight=avg_spw,
                eval_metric='logloss',
                random_state=RANDOM_STATE,
                n_jobs=-1,
                verbosity=0
            )
        )
        xgb_model.fit(X_train, y_train)
        print(f' Eğitim süresi: {time.time()-t0:.1f}s')
        save_model(xgb_model, 'xgboost_model_v2_1.pkl')
    except ImportError:
        print(' ⚠️ XGBoost yüklü değil, atlanıyor.')
        xgb_model = None
    except Exception as e:
        print(f' ⚠️ XGBoost hatası: {e}')
        xgb_model = None

if xgb_model is not None:
    xgb_metrics = evaluate_model(xgb_model, X_test, y_test, 'XGBoost')
    print(f' F1 Micro: {xgb_metrics["f1_micro"]:.4f} | F1 Macro: {xgb_metrics["f1_macro"]:.4f} | ROC-AUC: {xgb_metrics["roc_auc"]:.4f}')
```

🚀 Model 3: XGBoost

Dinamik scale_pos_weight: 98.5 (eski: sabit 10)
 Eğitim süresi: 3654.0s
 Kaydedildi: /content/drive/MyDrive/ToxicGuard/models/xgboost_model_v2_1.pkl
 F1 Micro: 0.3272 | F1 Macro: 0.4269 | ROC-AUC: 0.9904

🎯 BÖLÜM 7 — Threshold Optimizasyonu

Veri seti %90 zararsız → threshold=0.5 ile F1 düşük. F1-macro'yu maximize eden threshold'u buluyoruz.

```
# =====
# HÜCRE 13 – Optimal Threshold Bulma
# =====
def find_optimal_threshold(y_prob, y_true, thresholds=None):
    if thresholds is None:
        thresholds = np.arange(0.1, 0.7, 0.05)
    y_true_arr = np.array(y_true)
    best_thr, best_f1 = 0.5, -1
```

```

results = []
for thr in thresholds:
    y_pred = (y_prob >= thr).astype(int)
    f1 = f1_score(y_true_arr, y_pred, average='macro', zero_division=0)
    results.append((thr, f1))
    if f1 > best_f1:
        best_f1, best_thr = f1, thr
return best_thr, best_f1, results

print('🔍 Threshold Optimizasyonu...')
print('=' * 60)

optimal_thresholds = {}
model_list = [
    (lr_metrics, 'Logistic Regression'),
    (svm_metrics, 'Linear SVM'),
]
if xgb_metrics:
    model_list.append((xgb_metrics, 'XGBoost'))

for metrics, mname in model_list:
    y_prob = metrics['y_prob']
    best_thr, best_f1, _ = find_optimal_threshold(y_prob, y_test)
    optimal_thresholds[mname] = best_thr
    print(f'{mname}:')
    print(f'  threshold=0.50 → F1 macro: {metrics["f1_macro"]:.4f}')
    print(f'  threshold={best_thr:.2f} → F1 macro: {best_f1:.4f} (↑ {best_f1 - metrics["f1_macro"]:.4f})')
    print()

```

```

🔍 Threshold Optimizasyonu...
=====
Logistic Regression:
  threshold=0.50 → F1 macro: 0.6483
  threshold=0.65 → F1 macro: 0.7121 (↑ 0.0638)

Linear SVM:
  threshold=0.50 → F1 macro: 0.7593
  threshold=0.35 → F1 macro: 0.9188 (↑ 0.1595)

XGBoost:
  threshold=0.50 → F1 macro: 0.4269
  threshold=0.65 → F1 macro: 0.5583 (↑ 0.1314)

```

```

# =====
# HÜCRE 14 – Threshold Grafik
# =====
n_models = len(model_list)
fig, axes = plt.subplots(1, n_models, figsize=(6*n_models, 5))
if n_models == 1:
    axes = [axes]

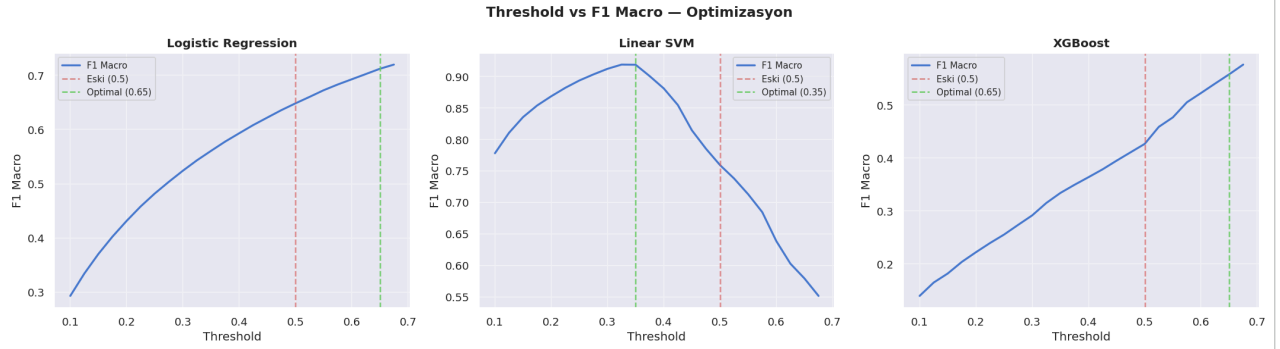
colors = ['#3498db', '#e74c3c', '#2ecc71']

for ax, (metrics, mname) in zip(axes, model_list):
    y_prob = metrics['y_prob']
    thrs = np.arange(0.1, 0.7, 0.025)
    f1s = []
    for thr in thrs:
        yp = (y_prob >= thr).astype(int)
        f1s.append(f1_score(np.array(y_test), yp, average='macro', zero_division=0))

    ax.plot(thrs, f1s, 'b-', linewidth=2, label='F1 Macro')
    opt = optimal_thresholds.get(mname, 0.5)
    ax.axvline(x=0.5, color='r', linestyle='--', alpha=0.7, label='Eski (0.5)')
    ax.axvline(x=opt, color='g', linestyle='--', alpha=0.9, label=f'Optimal ({opt:.2f})')
    ax.set_title(mname, fontweight='bold')
    ax.set_xlabel('Threshold')
    ax.set_ylabel('F1 Macro')
    ax.legend(fontsize=9)
    ax.grid(True, alpha=0.4)

plt.suptitle('Threshold vs F1 Macro – Optimizasyon', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig(os.path.join(RESULTS_DIR, 'threshold_optimization.png'), bbox_inches='tight')
plt.show()
print('✅ threshold_optimization.png')

```



✓ threshold_optimization.png

✖ BÖLÜM 8 — Final Değerlendirme

```
# =====
# HÜCRE 15 – Final Metrikler (Optimal Threshold)
# =====

# Model obje eşleştirme
model_objects = {
    'Logistic Regression': lr_model,
    'Linear SVM': svm_model,
}
if xgb_model is not None:
    model_objects['XGBoost'] = xgb_model

print('=' * 70)
print('FİNAL SONUÇLAR (Optimal Threshold ile)')
print('=' * 70)

all_results = []
for mname, model_obj in model_objects.items():
    opt_thr = optimal_thresholds.get(mname, 0.5)
    final = evaluate_model(model_obj, X_test, y_test, mname, threshold=opt_thr)
    all_results.append(final)

rows = []
for r in all_results:
    rows.append({
        'Model' : r['model'],
        'Threshold' : r['threshold'],
        'F1 Micro' : r['f1_micro'],
        'F1 Macro' : r['f1_macro'],
        'Precision' : r['precision_macro'],
        'Recall' : r['recall_macro'],
        'ROC-AUC' : r['roc_auc'] if r['roc_auc'] else 'N/A',
        'Accuracy' : r['accuracy'],
    })

comp_df = pd.DataFrame(rows).sort_values('F1 Macro', ascending=False).reset_index(drop=True)
display(comp_df)
```

=====

FİNAL SONUÇLAR (Optimal Threshold ile)

=====

	Model	Threshold	F1 Micro	F1 Macro	Precision	Recall	ROC-AUC	Accuracy
0	Linear SVM	0.35	0.9359	0.9188	0.8642	0.9812	0.9984	0.9751
1	Logistic Regression	0.65	0.8027	0.7121	0.5877	0.9706	0.9956	0.9256
2	XGBoost	0.65	0.4533	0.5583	0.4020	0.9728	0.9904	0.6092

Sonraki adımlar: [comp_df ile kod oluşturun](#) [New interactive sheet](#)

```
# =====
# HÜCRE 16 – Per-Label Rapor
# =====
print('=' * 70)
print('ETİKET BAZLI DETAYLI RAPOR')
print('=' * 70)

for r in all_results:
    print(f'\n📊 {r["model"]} (threshold={r["threshold"]:.2f}):')
    per_rows = []
    for col in LABEL_COLS:
        per_rows.append({
            'Etiket' : LABEL_TR[col],
            'F1' : r['per_label'][col]['f1'],
            'Precision': r['per_label'][col]['precision'],
            'Recall' : r['per_label'][col]['recall'],
        })
    display(pd.DataFrame(per_rows).set_index('Etiket'))
```

=====

ETİKET BAZLI DETAYLI RAPOR

=====

📊 Logistic Regression (threshold=0.65):

	F1	Precision	Recall	📊
Etiket				
Toksik	0.8526	0.8083	0.9021	
Ağır Toksik	0.5442	0.3739	0.9994	
Müstehcen	0.8611	0.7868	0.9510	
Tehdit	0.6224	0.4518	1.0000	
Hakaret	0.8106	0.6950	0.9723	
Kimlik Nefreti	0.5817	0.4104	0.9986	

📊 Linear SVM (threshold=0.35):

	F1	Precision	Recall
Etiket			
Toksik	0.9465	0.9090	0.9872
Ağır Toksik	0.8463	0.7879	0.9140
Müstehcen	0.9486	0.9069	0.9944
Tehdit	0.9183	0.8490	1.0000
Hakaret	0.9230	0.8610	0.9947
Kimlik Nefreti	0.9302	0.8717	0.9971

📊 XGBoost (threshold=0.65):

	F1	Precision	Recall
Etiket			
Toksik	0.3636	0.2225	0.9934
Ağır Toksik	0.5379	0.3711	0.9774
Müstehcen	0.6438	0.4831	0.9645
Tehdit	0.7642	0.6184	1.0000
Hakaret	0.4979	0.3352	0.9677
Kimlik Nefreti	0.5422	0.3820	0.9337

```
# =====
# HÜCRE 17 – Model Karşılaştırma Grafiği
# =====
fig, axes = plt.subplots(1, 3, figsize=(16, 6))

metrics_to_plot = [
    ('f1_macro', 'F1 Macro'),
    ('roc_auc', 'ROC-AUC'),
    ('recall_macro', 'Recall Macro'),
]
```

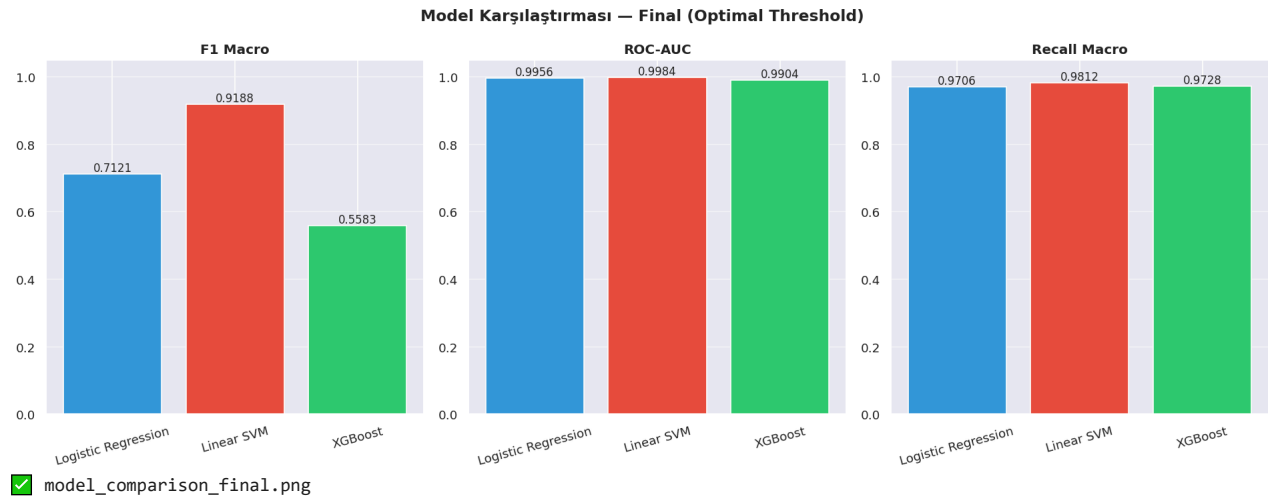
```

colors = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12']

for ax, (mkey, mlabel) in zip(axes, metrics_to_plot):
    vals = [r[mkey] if r[mkey] else 0 for r in all_results]
    names = [r['model'] for r in all_results]
    bars = ax.bar(names, vals, color=colors[:len(names)])
    ax.bar_label(bars, fmt='%.4f', fontsize=10)
    ax.set_title(mlabel, fontweight='bold')
    ax.set_ylim(0, 1.05)
    ax.tick_params(axis='x', rotation=15)
    ax.grid(axis='y', alpha=0.4)

plt.suptitle('Model Karşılaştırması — Final (Optimal Threshold)', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig(os.path.join(RESULTS_DIR, 'model_comparison_final.png'), bbox_inches='tight')
plt.show()
print('✅ model_comparison_final.png')

```



📁 BÖLÜM 9 — En İyi Modeli Seç & Kaydet

```

# =====
# HÜCRE 18 — En İyi Modeli Seç
# =====
best_result = max(all_results, key=lambda r: (
    r['f1_macro'],
    r['roc_auc'] if r['roc_auc'] else -1
))

best_name = best_result['model']
best_thresh = best_result['threshold']
best_model_obj = model_objects[best_name]

print(f'🏆 En iyi model: {best_name}')
print(f'F1 Macro : {best_result["f1_macro"]:.4f}')
print(f'ROC-AUC : {best_result["roc_auc"]:.4f}')
print(f'Threshold : {best_thresh:.2f}')

# best_model.pkl olarak kaydet (predict.py ile uyumlu)
save_model(best_model_obj, 'best_model.pkl')
# TF-IDF vectorizer da kaydet
save_model(tfidf, 'tfidf_vectorizer.pkl')

```

```
# Threshold config kaydet
threshold_config = {
    'best_model'      : best_name,
    'threshold'       : best_thresh,
    'opt_thresholds'  : optimal_thresholds,
    'training_notes'  : 'clean_text_v2_1 + optimize edilmiş threshold',
    'tfidf_file'      : 'tfidf_vectorizer.pkl',
    'model_file'      : 'best_model.pkl',
}
thr_path = os.path.join(MODELS_DIR, 'threshold_config.json')
with open(thr_path, 'w', encoding='utf-8') as f:
    json.dump(threshold_config, f, indent=2, ensure_ascii=False)
print(f'\n✅ threshold_config.json kaydedildi')
```

🏆 En iyi model: Linear SVM
 F1 Macro : 0.9188
 ROC-AUC : 0.9984
 Threshold : 0.35
 📁 Kaydedildi: /content/drive/MyDrive/ToxicGuard/models/best_model.pkl
 📁 Kaydedildi: /content/drive/MyDrive/ToxicGuard/models/tfidf_vectorizer.pkl

✅ threshold_config.json kaydedildi

```
# =====
# HÜCRE 19 – Sonuçları JSON & CSV Olarak Kaydet
# =====
# y_prob'u serializable listeden çıkar
serializable = []
for r in all_results:
    sr = {k: v for k, v in r.items() if k != 'y_prob'}
    serializable.append(sr)

json_path = os.path.join(RESULTS_DIR, 'model_comparison.json')
with open(json_path, 'w', encoding='utf-8') as f:
    json.dump({
        'metadata': {
            'best_model'      : best_name,
            'best_threshold' : best_thresh,
            'improvement'     : 'clean_text_v2_1 + threshold optimizasyonu',
        },
        'results': serializable
    }, f, indent=2, ensure_ascii=False)

csv_path = os.path.join(RESULTS_DIR, 'model_comparison.csv')
comp_df.to_csv(csv_path, index=False, encoding='utf-8-sig')

print(f'✅ JSON → {json_path}')
print(f'✅ CSV → {csv_path}')
print(f'\n🎉 Tüm işlemler tamamlandı!')
print(f'\n📊 Karşılaştırma Tablosu:')
display(comp_df)
```

✅ JSON → /content/drive/MyDrive/ToxicGuard/reports/model_results/model_comparison.json
 ✅ CSV → /content/drive/MyDrive/ToxicGuard/reports/model_results/model_comparison.csv

🎉 Tüm işlemler tamamlandı!

📊 Karşılaştırma Tablosu:

	Model	Threshold	F1 Micro	F1 Macro	Precision	Recall	ROC-AUC	Accuracy	
0	Linear SVM	0.35	0.9359	0.9188	0.8642	0.9812	0.9984	0.9751	🔗
1	Logistic Regression	0.65	0.8027	0.7121	0.5877	0.9706	0.9956	0.9256	
2	XGBoost	0.65	0.4533	0.5583	0.4020	0.9728	0.9904	0.6092	

Sonraki adımlar: [comp_df ile kod oluşturun](#) [New interactive sheet](#)

📌 BÖLÜM 10 — Hızlı Test (Canlı Tahmin)

```
# =====
# HÜCRE 20 – Canlı Tahmin Testi
# =====
test_sentences = [
    ('Thank you for your wonderful contribution!', 'ZARARSIZ'),
    ('I disagree but respect your view.', 'ZARARSIZ'),
    ('You are a complete idiot and should die!', 'TOKSİK'),
    ('I will find you and kill you, garbage!', 'TOKSİK'),
    ('Shut up you stupid moron!', 'TOKSİK'),
]
```

```
(("This is the worst article I've ever read.", 'TARTIŞMALI'),
 ("You're killing it! Amazing performance!", 'TARTIŞMALI (idiom)'),
]

print('=' * 70)
print(f'CANLI TAHMİN TESTİ - {best_name} (threshold={best_thresh:.2f})')
print('=' * 70)

for text, expected in test_sentences:
    cleaned = clean_text_v2_1(text)
```